

TEXT-TO-SPEECH APPLICATION

A Full-Stack Web Application for Converting Text into Natural-Sounding Speech

Abstract

The Text-to-Speech Application is a full-stack web application that converts written text into natural-sounding audio. The application allows users to enter text, select a supported language and voice, generate speech, play the generated audio, download it, and manage previously generated speech through History and Favorites sections. User authentication is implemented using Supabase Auth, while speech history is stored in a Supabase PostgreSQL database. The frontend is developed using React and Vite, and the backend is developed using Node.js and Express.js. Python with the edge-tts package is used to generate speech through Microsoft Edge online voices. The application is deployed using Vercel for the frontend and Render for the backend.

1. Introduction

Text is one of the most common forms of communication and information sharing. However, reading long passages may be difficult or inconvenient for some users. A Text-to-Speech system provides an alternative method of consuming information by converting written content into spoken audio. This project implements a web-based Text-to-Speech application with a responsive user interface and a backend service that handles speech generation, validation, authentication, and speech-history operations.

2. Problem Statement

Users may need to listen to written content while studying, multitasking, revising notes, or accessing information in different languages. Basic browser-based speech tools may not provide a unified interface for language selection, voice selection, audio download, user accounts, and saved speech history. The proposed application addresses this problem by providing a centralized platform where authenticated users can generate, play, download, and manage speech recordings.

3. Objectives

The main objectives of the project are:

- Convert user-provided text into speech.
- Provide language selection and male/female voice selection.
- Support multilingual speech generation, including English, Hindi, Marathi, and German where supported by the application.
- Provide audio playback with play/pause, volume, and seeking controls.
- Allow users to download generated audio.
- Implement user registration and login.
- Store user-specific speech history and favorites.
- Provide validation and error handling for invalid input and backend failures.
- Deploy the frontend and backend so that the application can be accessed online.

4. Scope of the Project

The scope of this project includes text input, character counting, language selection, voice selection, speech generation, audio playback, audio download, authentication, history management, favorite management, deletion of history records, and deployment. Advanced features such as PDF/DOCX extraction, AI text rewriting, speech speed controls, pitch controls, and administrator analytics are outside the current implementation scope.

5. Existing System and Proposed System

Existing solutions may provide speech generation through browser APIs or isolated online tools, but they may not combine multilingual selection, account-based access, saved history, favorites, and a dedicated full-stack architecture in one application.

The proposed system provides a React-based interface, a Node.js/Express backend, a Python speech-generation service, Supabase authentication, and a Supabase database. This architecture separates frontend presentation, backend processing, speech generation, and persistent user data.

6. Technology Stack

Frontend:

- React.js with Vite
- JavaScript and JSX
- Tailwind CSS / responsive CSS styling
- Axios and Fetch API

Backend:

- Node.js
- Express.js
- CORS
- Helmet.js
- Express JSON body parser

Speech Generation:

- Python
- edge-tts package
- Microsoft Edge online speech voices

Database and Authentication:

- Supabase Auth
- Supabase PostgreSQL database
- Row Level Security policies for speech-history records

Deployment and Development Tools:

- Visual Studio Code

- Git and GitHub
- Vercel for frontend deployment
- Render for backend deployment
- npm and Python package installation

7. System Architecture

The application follows a client-server architecture:

User → React/Vite Frontend → Node.js/Express REST API → Python edge-tts Service → Audio Response → Frontend Audio Player/Download

For authenticated data:

User → Frontend Login → Express Authentication Route → Supabase Auth

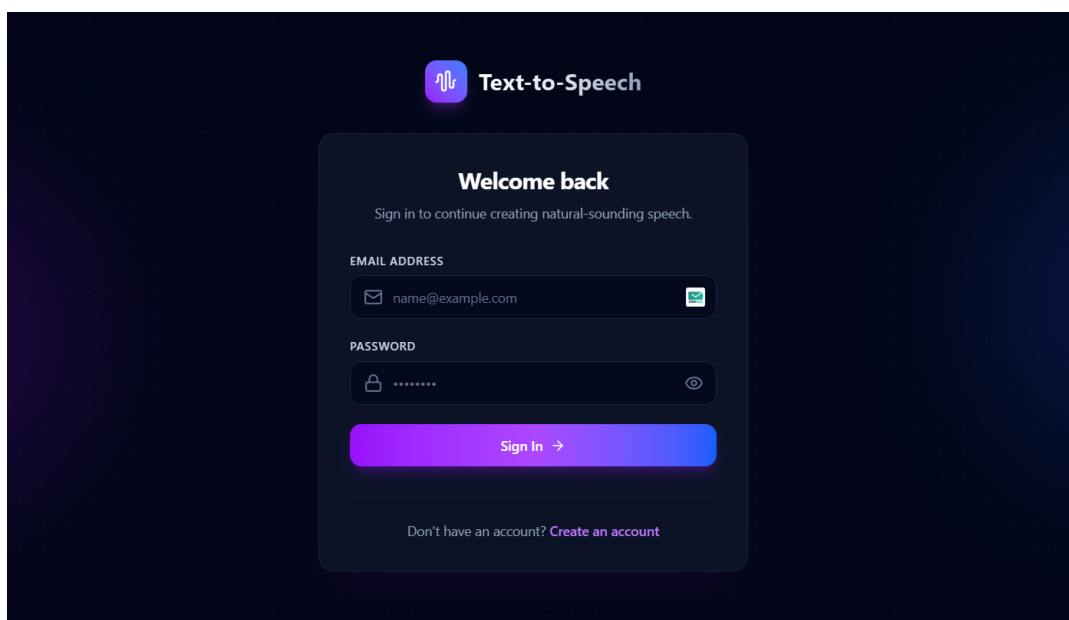
User → Frontend History Request → Authenticated Express Route → Supabase PostgreSQL Database

The frontend is responsible for user interaction and presentation. The backend validates requests, communicates with the speech-generation service, handles authentication tokens, and performs database operations. Supabase ensures that users can access only their own speech-history records through authentication and Row Level Security.

8. Major Modules

8.1 Authentication Module

Users can register using their name, email, and password and can log in using their registered credentials. The backend communicates with Supabase Auth. After successful login, the frontend stores the session access token for authenticated API requests.





Text-to-Speech

Create your account

Start creating and saving natural-sounding AI speech today.

FULL NAME

EMAIL ADDRESS

PASSWORD

CONFIRM PASSWORD

Create Account →

Already have an account? [Sign in](#)

8.2 Text Input and Validation Module

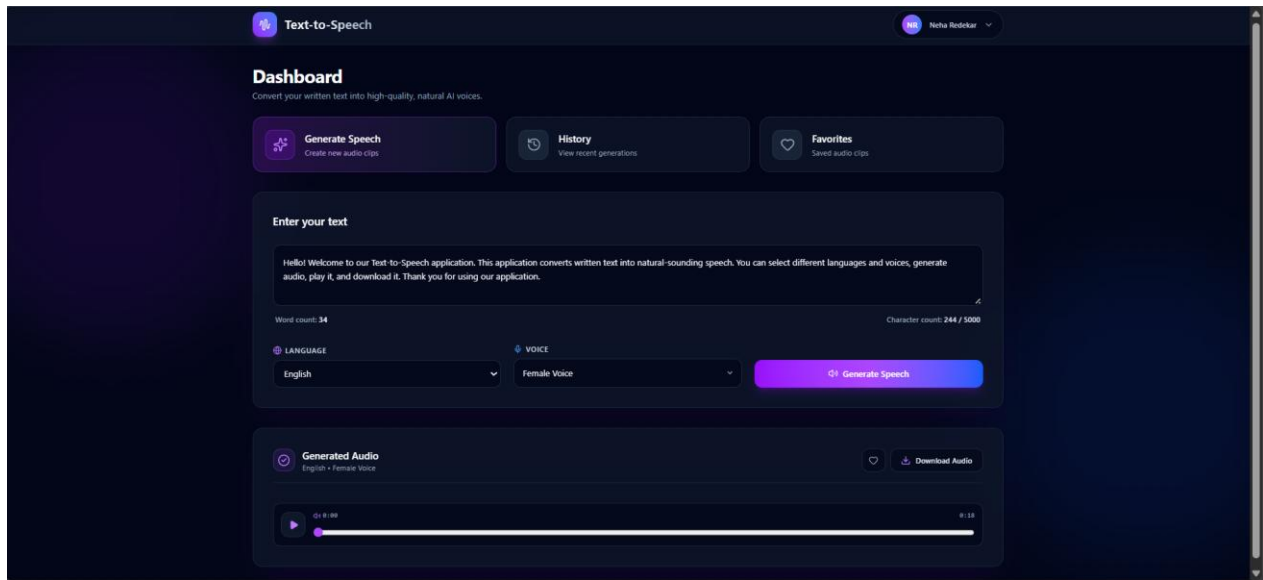
The user enters or pastes text into the text area. The interface displays character information and validates empty input, maximum length, supported language, and voice-related conditions.

8.3 Language and Voice Selection Module

The user can select a supported language and a male or female voice. The selected configuration is sent to the backend for speech generation.

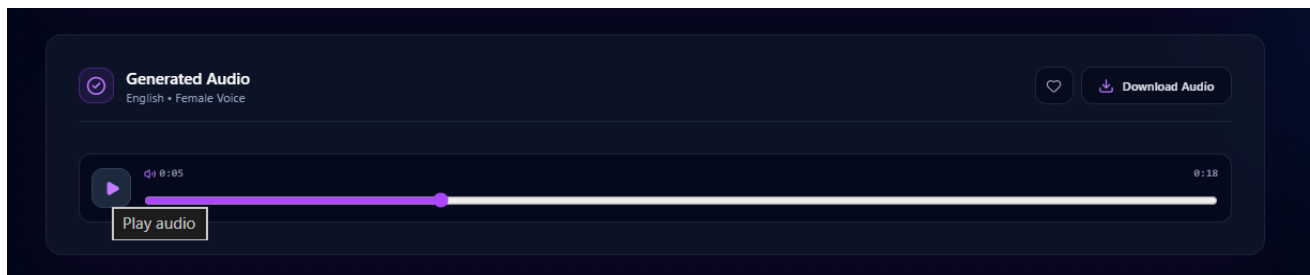
8.4 Speech Generation Module

The frontend sends a POST request to the backend. The backend validates the request and invokes the Python edge-tts service. The generated audio is returned to the frontend.



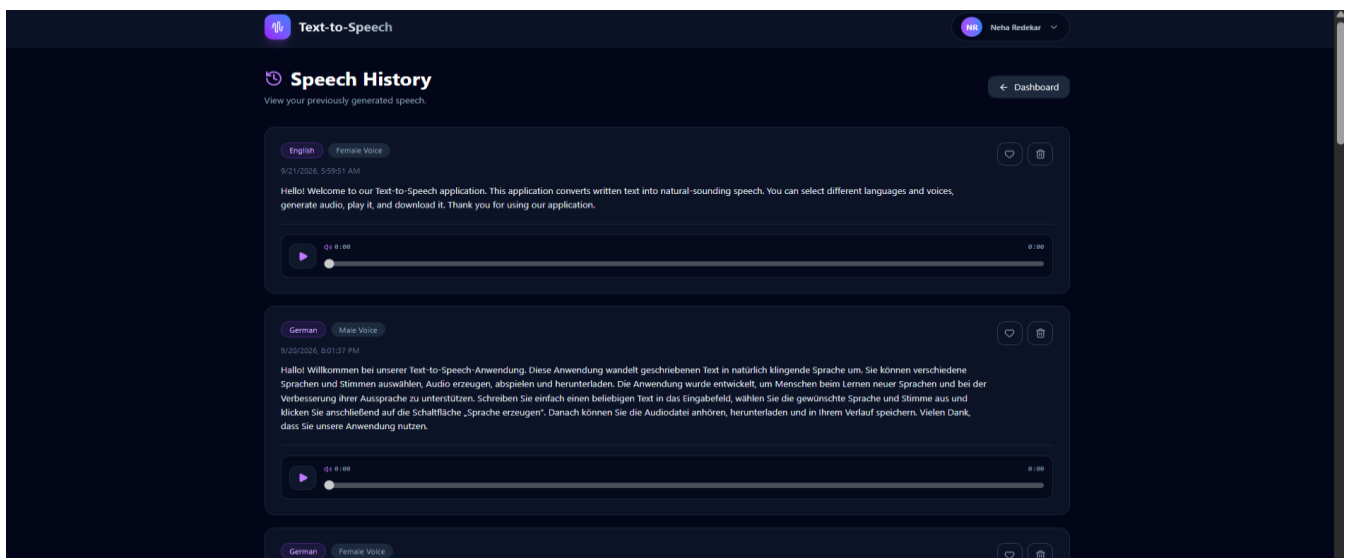
8.5 Audio Player and Download Module

The generated audio can be played in the browser. Users can pause, seek through the audio, adjust volume, and download the audio file.



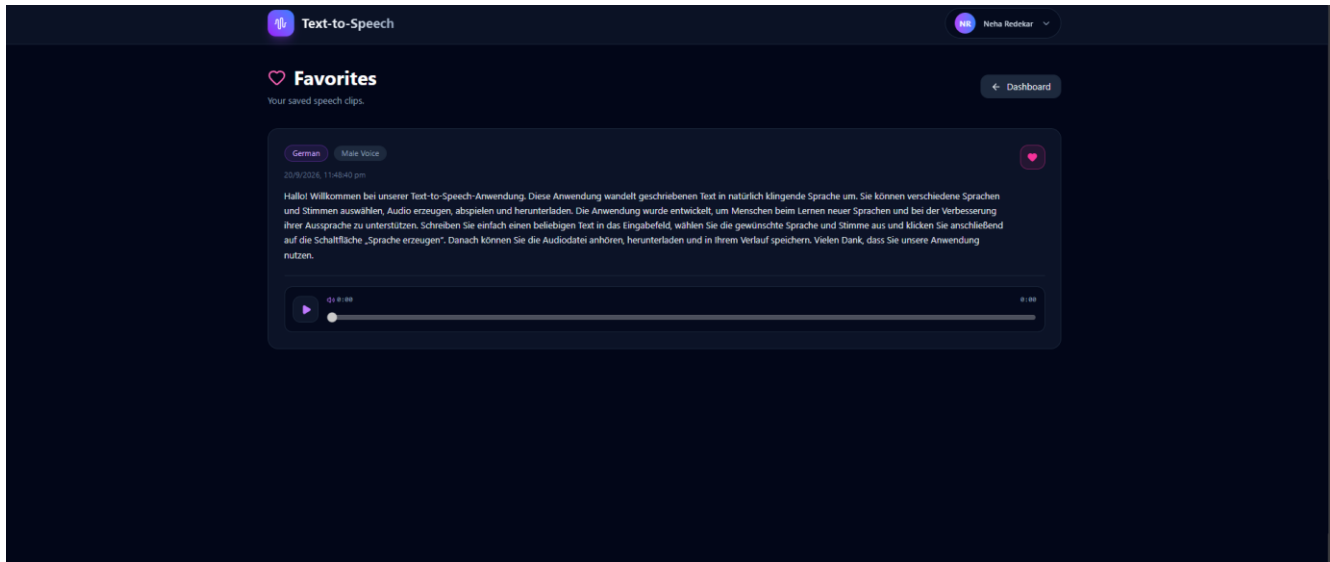
8.6 History Module

Generated speech records are stored in Supabase. The history section displays text, language, voice, date, and audio. Audio remains available after refreshing the application when the record is loaded from the database.



8.7 Favorites Module

Users can mark generated speech as a favorite. Favorite status is stored in the database and can be changed through the backend API.



8.8 Delete Module

Users can delete speech-history records. Database operations are restricted to the authenticated user's own records.

9. Database Design

The application uses a Supabase PostgreSQL table named `speech_history`. Important fields include:

- `id`: UUID primary key
- `user_id`: UUID referencing the authenticated Supabase user
- `text`: Generated text content
- `language`: Selected language
- `voice`: Selected voice
- `audio`: Stored audio data or audio representation used by the application
- `date`: Display date of the speech record
- `created_at`: Record creation timestamp
- `is_favorite`: Boolean value representing favorite status

Row Level Security is enabled. Policies allow authenticated users to select, insert, update, and delete only records where `user_id` matches `auth.uid()`.

10. Important API Endpoints

Authentication:

- `POST /api/auth/signup` – creates a user account.
- `POST /api/auth/login` – authenticates a user and returns session information.

Text-to-Speech:

- GET /api/health – checks backend health.
- POST /api/tts – receives text, language, and voice and generates audio.

Speech History:

- GET /api/history – retrieves the authenticated user's history.
- POST /api/history – saves a generated speech record.
- PATCH /api/history/:id/favorite – toggles favorite status.
- DELETE /api/history/:id – deletes a history record.

Authenticated history endpoints require a Bearer access token.

11. Working Procedure

1. The user opens the application and registers or logs in.
2. The user enters text in the dashboard.
3. The user selects a language and voice.
4. The frontend validates the input and sends a request to the Express backend.
5. The backend validates the request and calls the Python edge-tts service.
6. The speech service generates an audio file or audio response.
7. The backend returns the generated audio to the frontend.
8. The frontend displays the audio player and download option.
9. The generated record is saved to Supabase for the authenticated user.
10. The user can view the record in History, mark it as a Favorite, or delete it.

12. Validation, Security, and Error Handling

The application includes frontend and backend validation for empty text, text length, supported languages, and supported voices. API errors are displayed to the user through error messages. The backend uses CORS configuration, Helmet security headers, JSON request-size limits, and authentication middleware. Supabase credentials are stored in environment variables and are not written directly into frontend source code. The .env file must not be committed to GitHub. Row Level Security policies protect user-specific history data.

13. Testing

The application was tested through functional checks of the major features:

- User registration and login
- Separate user account access
- Text input and character count

- English, Hindi, Marathi, and German speech generation where supported
- Male and female voice selection
- Audio playback
- Audio seeking and volume controls
- Audio download
- History loading after refresh
- Favorites creation and removal
- History deletion
- Handling of long text requests
- Handling of backend and TTS errors
- Frontend-to-backend communication after deployment

During development, a request-size issue for long text was addressed by configuring the Express JSON body parser with a larger limit. A deployment issue related to the Python edge-tts dependency was addressed by including Python dependency installation in the Render build command.

14. Deployment

The frontend is deployed on Vercel and the backend is deployed on Render. The backend build process installs Node.js dependencies and Python dependencies using the requirements file. The backend start command runs the Node.js server. Environment variables for Supabase and other required configuration values are added through the deployment platform's environment settings. The frontend uses the deployed backend URL instead of the local development URL when running online.

15. Results

The completed application provides a working web interface for converting written text into speech. Users can authenticate, select a language and voice, generate audio, play and download the result, and manage speech records through History and Favorites. The frontend and backend are deployed separately, and the backend communicates with Supabase for authentication and persistent speech-history data.

16. Future Enhancements

Possible future improvements include:

- Upload and conversion of TXT, PDF, and DOCX files.
- Speech speed, pitch, and volume customization.
- Additional languages and regional accents.
- AI-based grammar correction, summarization, and text enhancement.
- Cloud object storage for audio files.
- Usage limits and analytics.
- Improved accessibility features.
- Automated unit, integration, and end-to-end testing.
- Progressive Web App support for mobile users.

17. Conclusion

The Text-to-Speech Application demonstrates the development of a complete full-stack web application using React, Node.js, Express.js, Python, Supabase, GitHub, Vercel, and Render. It combines speech generation with authentication and user-specific history management. The project provides practical experience in REST API communication, third-party service integration, database operations, authentication, validation, security, error handling, and cloud deployment.